

AZURE DATA ENGINEERING PROJECT

# On-Prem to ADLS Gen2 Migration

A metadata-driven incremental data pipeline built with Azure Data Factory —  
SQL Server → Azure Data Lake Storage Gen2, orchestrated end to end.

20

Tables Migrated

3

Orchestrated Pipelines

100%

Metadata-Driven

# Why this pipeline exists

- Operational data was trapped on a single on-premise SQL Server instance — no scalable way to run analytics or BI on top of it.
- The goal: move that data into Azure Data Lake Storage Gen2, where it can feed dashboards, warehouses, and downstream analytics at scale.
- Copying everything, every time, doesn't scale — so the pipeline had to be incremental: only new or changed rows move on each run.
- The pipeline also had to be metadata-driven: one generic set of logic that works for 20 tables today, and any number tomorrow — without hand-building a pipeline per table.

## THE SHIFT

### On-Premise SQL Server

Local network only • 20 tables • star schema



*Self-hosted Integration Runtime  
bridges the network gap*



### Azure Data Lake Storage Gen2

Parquet files • dated folders • ready for analytics

# Three pipelines, one orchestrated flow



Each box below is a separate ADF pipeline — built once, reused for every one of the 20 tables via parameters.

# The Azure foundation, piece by piece



## Resource Group

One folder holding every resource — for cost tracking, management, and clean teardown.



## ADLS Gen2 Storage

Hierarchical-namespace storage account — the real destination. Input/output containers hold dated, per-table folders.



## Data Factory

The orchestration engine. Runs Lookup, ForEach, IF, Copy, and Stored Procedure activities on demand or on a schedule.



## Key Vault

Secure secret store for the SQL password — so the credential never sits as plain text in Git-tracked JSON.



## Self-hosted Integration Runtime

A local agent bridging the network gap — lets cloud ADF reach an on-prem SQL Server with no public exposure.

# Why the password never touches Git

## ✗ Hardcoded in the Linked Service

- Password sits in plain text inside pipeline JSON
- That JSON is committed straight to a public GitHub repo
- Anyone with repo access can read the live SQL credential

## ✓ What this project does instead

- Password stored once, as an Azure Key Vault secret
- SQL Server Linked Service references the secret by name only
- ADF's managed identity is granted Key Vault Secrets User — read-only

## SECRET RESOLUTION AT RUNTIME

### LS\_OnPremSQL\_Batch10 needs a password

Points to Key Vault, not a literal string

### Managed identity authenticates to Key Vault

No username/password — identity is the credential

### Secret 'SqlServerPassword' is fetched

# The reusable ‘what am I connecting to’

*A Linked Service is the phone line. A Dataset is the number you actually dial on it.*

## **DS\_Watermark\_Table**

Fixed — always `dbo.ADF_Watermark_Table`

Read once at the start by the Parent pipeline to get the full list of tables and their last-loaded values.

## **DS\_LKUP\_MAX\_VALUE**

Parameterized — `TableSchema, TableName`

Reused every loop iteration to point at a different source table — used for both the MAX-value check and the Copy source.

## **DS\_ADLS\_Sink**

Parameterized — `TableName`

Builds a dynamic Parquet file path per table: dated folder + timestamped filename, so nothing is ever overwritten.

## PL\_Update\_Watermark — closing the loop

- Purpose: after a table's data has been copied, record how far the load got — so the next run only picks up what changed after that point.
- Built as its own pipeline (not inline logic) because both other pipelines need to call it — build once, reuse everywhere.
- Accepts 3 parameters: TableSchema, TableName, WatermarkColumn — passed straight through from whoever calls it.
- Single activity: Stored Procedure, calling the existing `dbo.UpdateWatermark` on the SQL Server — not a Copy or Lookup, because its whole job is to execute logic, not move or read data.

### WHAT `dbo.UpdateWatermark` DOES

1. Build dynamic SQL:  

```
SELECT MAX([WatermarkColumn])  
FROM [TableSchema].[TableName]
```
2. Run it → get the current highest key value
3. `ISNULL(..., 0)` → handle an empty table safely
4. UPSERT into `dbo.ADF_Watermark_Table`:
  - row exists → UPDATE
  - row missing → INSERT

# PL\_Incremental\_Load — deciding whether to move data

L

## Activity 1 — Lookup

Its one job: run a read-only query and hand the result to the next activity — it never writes anywhere.

```
SELECT MAX(WatermarkColumn)
AS MaxValue
FROM TableSchema.TableName
```

- “First row only” ticked — unwraps the array for easy reference downstream

IF

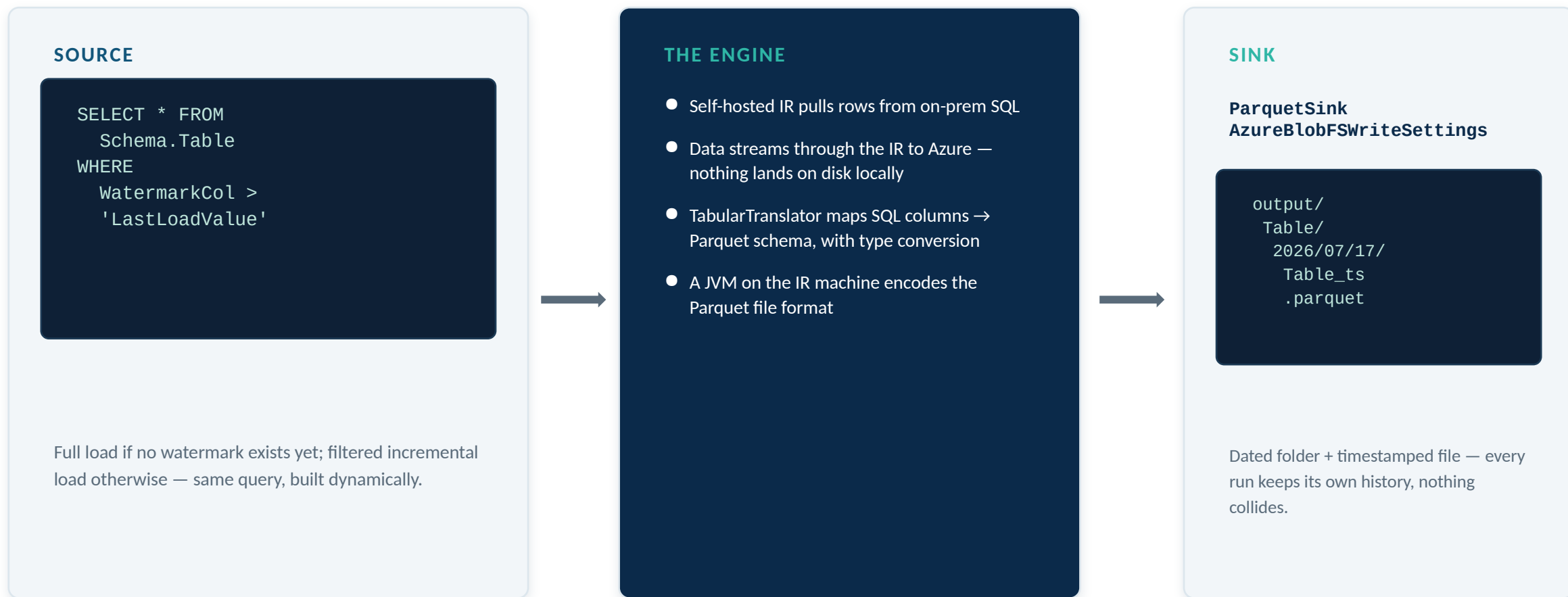
## Activity 2 — If Condition

Gatekeeper: only copy data when it's genuinely needed, avoiding wasted compute and empty files.

```
@or(
  equals(LastLoadValue, null),
  greater(
    int(MaxValue),
    int(LastLoadValue)
  )
)
```

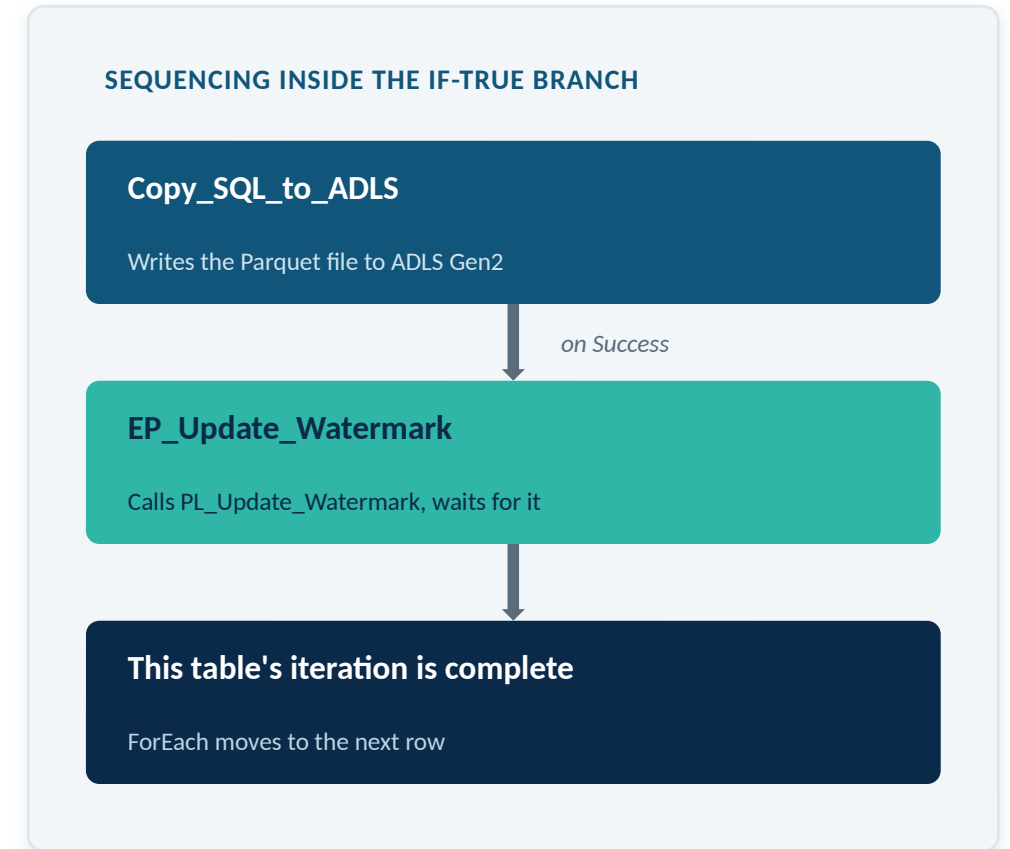
*True when: this is the very first load, OR the source has new data past the last checkpoint.*

# Copy\_SQL\_to\_ADLS — what happens behind the scenes



## EP\_Update\_Watermark — finishing the iteration

- Activity type: Execute Pipeline — calls PL\_Update\_Watermark, passing TableSchema, TableName, and WatermarkColumn through.
- Connected with a success dependency to Copy\_SQL\_to\_ADLS — it only fires after the copy genuinely succeeds.
- “Wait on completion” is ticked — the pipeline treats this table as fully done only once the watermark is actually written back.
- **Without this ordering, the watermark could update before the copy finishes — silently marking data as loaded that never actually arrived.**



# PL\_Parent\_Orchestrator — the front door



## Activity 1 — Lookup

Reads the entire dbo.ADF\_Watermark\_Table — every row, not just one.

- Source type: Table (not a filtered Query) — the whole control list is needed
- “First row only” left unticked — this time every row matters
- Output: an array of 20 rows, each with TableSchema, TableName, WatermarkColumn, LastLoadValue



## Activity 2 — ForEach

Loops over the Lookup's output — once per table, calling the incremental-load pipeline each time.

```
Items: @activity('LKUP_Watermark_Table')  
        .output.value
```

- Sequential ticked — one table fully finishes before the next starts
- Inside: Execute Pipeline → PL\_Incremental\_Load, parameters filled from @item()
- @item() = the current row this specific loop pass is processing

# From a single click to twenty Parquet files

- 1 Trigger fires (or Debug run starts) — PL\_Parent\_Orchestrator begins
- 2 Lookup reads all 20 rows from dbo.ADF\_Watermark\_Table
- 3 ForEach begins, sequentially, table by table
- 4 PL\_Incremental\_Load runs — Lookup gets the source table's current MAX value
- 5 IF Condition checks: is this a first load, or is there genuinely new data?
- 6 If true — Copy activity streams rows from SQL Server through the Self-hosted IR into a dated, timestamped Parquet file in ADLS Gen2
- 7 PL\_Update\_Watermark runs — recalculates and writes the new LastLoadValue
- 8 ForEach advances to the next table — repeat until all 20 are processed

VALIDATION

# Proof it actually works — not just green checkmarks

A green “Succeeded” in ADF confirms the activity ran without error — the real proof is in the data. Querying the watermark table directly after the run confirmed every step of the chain.

| Table                  | Watermark Column | Before | After Run     |
|------------------------|------------------|--------|---------------|
| DimCustomer            | CustomerKey      | 0      | 10,000        |
| DimProduct             | ProductKey       | 0      | 1,000         |
| DimStore               | StoreKey         | 0      | 50            |
| ... and 17 more tables | —                | 0      | fully updated |

ROOT-CAUSE FIX

Parquet writes failed with JreNotFound

Parquet encoding runs through a JVM on the Self-hosted IR machine — it wasn't installed. Installing Microsoft OpenJDK and restarting the IR service resolved it on the next run.

## KEY TAKEAWAYS

# What this project demonstrates

1

### Metadata-driven design

One pipeline, one set of logic, driven entirely by a control table — scales to any number of tables with zero code changes.

2

### True incremental loading

Watermark-based checkpoints mean every run moves only what changed — no wasted compute, no duplicate data.

3

### Security by default

Credentials never leave Key Vault; ADF authenticates with a managed identity, not a password in code.

4

### Fully version-controlled

Every pipeline, dataset, and linked service is Git-backed — clean history, safe collaboration, reviewable changes.